BRAINSTER

Objects 1

Brainster Web Development Academy

Table of contents

Object basics

- An object is a collection of related data and/or functionality (which usually consists of several variables and functions — which are called properties and methods when they are inside objects.)
- In terms of JavaScript, and object is also a type, with its own properties and methods.
- Also, everything is an object in JavaScript (not literally, but close to it).

Object basics

- In JavaScript we can define an object in the following manner:

  const myObject = {}; // quite boring by itself

- However, as we mentioned, an object can contain variables:

  const myObject = {
  someNumber: 5 // this is similar as writing: const someNumber = 5;
  }

Object basics

- The only difference opposed to regular variables, is the need to use the ":" sign to assign value, instead of the regular "=". We also don't need var, let or const.

- A JavaScript object can also contain multiple variables:

```
const myObject = {
        someNumber: 5,  // note the "," here
        someText: "hello"
}
```

- So, to have multiple variables in an object, we need to separate them by colons (,).

- This, of course goes for every type of variable - numbers, text, arrays - even HTML elements.

Exercise 1

1. Define an object called “Cat”.
2. The “Cat” object should also contain two variables (properties) called “name” and “color”. Set some values to them (for example: “Tom” and “blue”).

Dot notation

- Just as we can define variables inside an object, we can access them just as easily:

```
const myObject = {
        someNumber: 5,
        someText: “hello”
};

console.log(myObject.someNumber) // 5
console.log(myObject.something) // undefined
```


Dot notation

- It's not out first time encountering the use of the "." symbol. Officially, using it to access properties (variables) of an object is called "dot notation".

- This has another, not so obvious benefit - it allows us to change the values of the properties (variables) that belong to an object.

```
const myObject = {
        someNumber: 5,
        someText: "hello"
};

myObject.someNumber = 6;
myObject.someText = "goodbye";

console.log(myObject); // {someNumber: 6, someText: "goodbye"
```


Bracket notation

- The Bracket Notation approach involves using square brackets, in which you have an expression that evaluates to a value. That value serves as a key for accessing the property.

  object[expression]

- The expression within the brackets evaluates to a key for the property you want to access, and this expression will return the value of the property.


```
const myObject = {
        someNumber: 5,
        someText: “hello”
};

console.log(myObject[“someNumber”]) // 5
console.log(myObject[“something”]) // undefined
```

Bracket notation

- Similar to the dot notation, we can use bracket notation to add and modify properties to an object.

```
const myObject = {
        someNumber: 5,
        someText: “hello”
};

myObject[‘someNumber’] = 6;
myObject[‘someText’] = “goodbye”;

console.log(myObject) // {someNumber: 6, someText: “goodbye”}
```


Exercise 2

1. Change the values of the cat object from the previous exercise to something else, using dot notation.
2. Try the same thing using bracket notation.

Functions as object properties

- As we mentioned in the first slide, an object can also have functions as it's members (also called methods).

```
const myObject = {
        someFunction: function() { // only the variable definition style works
                console.log("I am a function inside an object");
        }
}

myObject.someFunction(); // "I am a function inside an object"
```


Functions as object properties

- Otherwise, all the regular rules that apply to functions, apply here as well - we can have input arguments, we can have return statements etc.
- The only significant difference is the way we define them - we can only define them in the “variable definition” style, opposed to the regular one.
- Note: arrow functions can also be used as object methods.

Exercise 3

1. Add a “meow” method to the Cat object. It should simply alert “meow” on the screen when invoked.
2. Invoke the “meow” method from the Cat object.

Adding data to an object

- Just as we could change some data on an object, we can also add both properties (variables) and methods (functions) to an existing object.

```
const myObject = {};

// add a new property
myObject.someNumber = 5;
myObject["someOtherNumber"] = 6;

// add a new method
myObject.someFunction = function () { console.log("hello"); }
myObject["someOtherFunction"] = function () { console.log("hello"); }
```


Changing data in an object

- Methods can also access and manipulate the data object. The most common way of doing it by using the "**this**" keyword.

```
const dog = {
        name: "", // name doesn't have to be here though!
        setName: function(nameArg) {
                this.name = nameArg; // "this" refers to dog here.
        }
}
```


Aside: About “this”

- “this” is simply a convenient way of accessing the object that the method (function) operates on. In the previous example, “this” was in fact “dog”.
-
- We can also use this approach to add new data to an object by simply using:

  this.someNewValue = “new value”;

  and the object would get a someNewValue property as a result.

Exercise 4

1. Define a “cat1” object.
2. Define a method called setName, which receives a string as an input argument and sets that string as the cat’s name. (hint: use the “this” keyword)
3. Define a method called setColor, which receives a string as an input argument and sets that string as the cat’s color (hint: use the “this” keyword)
4. Define a method called sayNameAndColor which alerts on screen the name and color of the cat.
5. Invoke all methods: setName, then setColor, then sayNameAndColor withdata of your choice.

Aside: Objects and other types

- For all intents and purposes, and object behaves as any other type (it has properties, methods...)
- As such, we can define arrays of objects too! All the regular rules for those apply there too.
- Additionally, objects can be nested inside objects.

example:

```
const cities = {
        Bitola: {
                population: 80000,
        },
        Skopje: {
                population: 500000,
        },
        Ohrid: {
                Population: 60000
        }
}
```


Objects methods

- Sometimes, we will need to show all the data from an object somehow. With an array, it is simple: loop through it and do something on every iteration.
- Luckily, we have something similar in objects!
- The Object class (on this later), has methods that help us loop through an object:

  - **Object.keys(obj)** – returns an array of keys.
  - **Object.values(obj)** – returns an array of values.
  - **Object.entries(obj)** – returns an array of [key, value] pairs.

```
const myObject = {
        someNumber: 5,
        someText: 'hello'
};

const keys = Object.keys(myObject); // ['someNumber', 'someText']
const values = Object.values(myObject); // [5, 'hello']
const entries = Object.entries(myObject); // [ ['someNumber', 5], ['someText', 'hello'] ]
```


Exercise 5

1. In HTML, create a table with two columns: City and Population.
2. Using the cities object from our example (you can add your city as well), fill the table with as many rows as there are objects in the object.
    a. Try it first with Object.keys and Object.values
    b. Try it again with Object.entries

Exercise 6

1. Create an empty array called “dogs”.
2. (In html) create two input fields (with different ids) and a button that says “Add dog”.
3. Create a function which:
    a. Gets the values from the two input fields
    b. Creates a new “dog” object using the first value as name and the second as color.
    c. Adds the newly created dog object to the array.
    d. Logs in the console the whole “dogs” array
4. Invoke the function when the button gets clicked (using “click” with addEventListener)

Exercise 7

1. Building on the previous exercise:
   a. Add an html table to the document with columns for “name” and “color”.
   b. Any time a new dog gets added to the array, create a new row in the table (using JavaScript) and show its name and color in the appropriate columns.

Homework

Expand the previous exercise further:

1. Add an extra column to the table, which contains a “delete” button for ever dog that exists in the table.
2. Create a function which deletes the dog record associated to the button.
   Hint: you can use Array.splice to do this quick (although there are other ways)